

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

Lernziele

- Kennenlernen der Datenbankadministrationssprache
- Wie lassen sich Funktionen und Prozeduren erstellen?
- Was ist der Unterschied zwischen einer normalen Ausgabe und einem Cursor?
- Welche Mittel gibt es gegen mögliche Fehler?
- Wie kann sichergestellt werden, dass Abfolgen von Anweisungen im Paket ausgeführt werden?
- Was versteht man unter Datenintegrität, welche Typen gibt es und wie lässt sich diese sicherstellen?
- Gibt es Möglichkeiten auf Ereignisse in der Datenbank zu reagieren?

Wiederholung

3 Gruppen von SQL-Anweisungen:

DDL Data Definition Language

- Verwaltung von Metadaten
- Bsp.: **CREATE, DROP, ALTER**

DML Data Manipulation Language

- RUDI (Read, Update, Delete, Insert):
- **SELECT, UPDATE, DELETE, INSERT**

DCL Data Control Language

- Verwaltung von Benutzern und Zugriffsrechten
- **CREATE USER, GRANT, REVOKE**

Übersicht

- 1 SQL (weitere Konzepte)
 - Datenbankadministrationssprache
 - Prozeduren und Funktionen
 - Cursors
 - Behandlung von Ausnahmen
 - Transaktionen
 - Datenintegrität
 - Trigger



Datenbankadministrationssprache

- Beläuft sich im Wesentlichen auf die folgenden Funktionen:
 - Erstellen / Löschen von Datenbank-Benutzern
 - Gewähren von Rechten innerhalb des DBMS
 - Entziehen dieser Rechte

Anlegen von Benutzern - CREATE USER

Deklaration (nicht vollständig)

```
1 CREATE USER '<user_name>'@ '<host>'
2 IDENTIFIED BY '<password>';
```

- Angabe von Benutzernamen und Passwort
- **host** gibt an, über welches Netzwerk sich Benutzer anmelden können.
 - Bspw.: **root** bestenfalls nur über localhost.
 - '192.168.41.%': Nur ein bestimmtes Subnetz
 - % : uneingeschränkt
- Weitere Angaben:
 - Authentifizierungsoptionen: Plugin, Random-Passwort...
 - Verschlüsselte Anmeldung
 - Ressourcen-Optionen: Maximale Anfragen/Std, Updates/Std, Verbindungen/Std, Benutzerverbindungen
 - Passwortoptionen: Ablauf, Historie, Anzahl Fehlversuche, ...
 - ...



- **Autorisierung** schränkt den Zugriff und die Änderung von Daten durch Benutzer ein.
- Beschränkungen können sich beziehen auf:
 - Objekte wie z. B. Schemas, Tabellen, Spalten, Zeilen
 - Ressourcen wie z. B. Zeit (CPU, Verbindung, oder Wartezeiten).
- Es gibt **Zugriffsrechte** auf verschiedenen Ebenen:
 - System: tablespace, ...
 - Schema: Cluster, Index, Trigger, Datenbank, ...
 - Tabellen: create, alter, index, references, drop, select, delete, insert, update, ...
 - View: create, select, delete, insert, update
 - Prozeduren: create, alter, drop
 - Typen: create, alter, drop, execute
- Zugriffsrechte können an **Benutzer oder Rollen** (Role Based Access Control) vergeben werden

Der GRANT Befehl

- Zuweisen von Rechten und Rollen an Benutzer sowie Benutzerrollen

Deklaration (nicht vollständig)

```
1 GRANT <Liste von Zugriffsrechten>
2   ON [Tabelle oder View] privilegtiefe
3   TO <Liste von Benutzern>
4   [WITH GRANT OPTION]
5 ;
```

- <Liste von Benutzern> kann sein:
 - ein Benutzername
 - eine Rolle
 - **public**: alle gültigen Benutzer
- Zugriffsrechte auf Views bewirken keine Zugriffsrechte auf die dazugehörigen Basistabellen.
- Vergeber eines Zugriffsrechtes müssen dieses selber besitzen (oder Administrator sein).

Einige Zugriffsrechte auf Tabellen

SELECT: Direktes Leserecht über select-Anfragen oder indirektes Leserecht über Views.

- Beispiel: **SELECT** Zugriffsrecht für Benutzer U_1 , U_2 und U_3 auf Relation Filialen vergeben:

GRANT SELECT ON Filialen **TO** U_1 , U_2 , U_3 ;

INSERT: erlaubt Einfügen von Zeilen mit dem **INSERT** Befehl

UPDATE: erlaubt Ändern von Werten mit dem **UPDATE** Befehl

DELETE: erlaubt Löschen von Zeilen mit dem **DELETE** Befehl (**DROP TABLE** ist jedoch nicht erlaubt!)

Der REVOKE Befehl

- Der revoke Befehl nimmt Zugriffsrechte zurück.

Deklaration (nicht vollständig)

```
1 REVOKE <Liste von Zugriffsrechten>
2 ON [Tabelle oder View] privilegtiefe
3 FROM <Liste von Benutzern>
4 [WITH GRANT OPTION]
5 ;
```

- Beispiel:

REVOKE SELECT ON Filialen FROM U_1, U_2, U_3

- <Liste von Zugriffsrechten> kann **ALL** sein, um alle Zugriffsrechte zurückzunehmen.
- Falls <Liste von Zugriffsrechten> **PUBLIC** enthält, verlieren alle Benutzer die angegebenen Rechte, außer die Rechte wurden explizit gewährt.
- Falls dasselbe Zugriffsrecht von zwei verschiedenen Benutzern gewährt wurde, kann es auch nach dem **REVOKE** Befehl erhalten bleiben.

Übersicht

1 SQL (weitere Konzepte)

Datenbankadministrationssprache

Prozeduren und Funktionen

Cursors

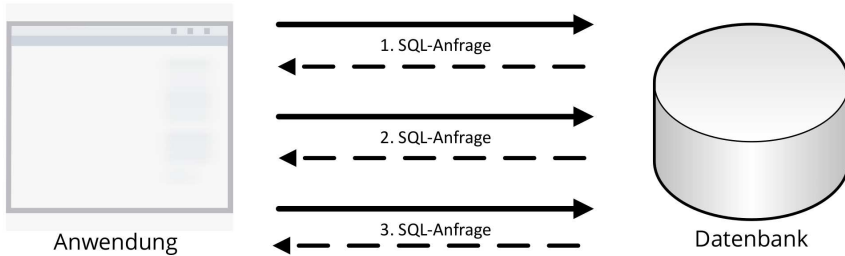
Behandlung von Ausnahmen

Transaktionen

Datenintegrität

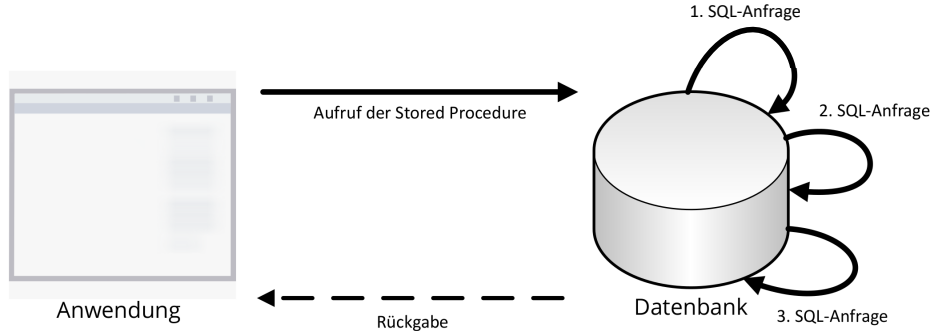
Trigger

Stored Procedures/Functions (Ohne)



- Manche Aufgaben lassen sich nicht in einem SQL-Befehl unterbringen.
- Oft mehrere SQL-Befehle innerhalb einer Transaktion nacheinander abgearbeitet.
- Einzeln versenden über das Netzwerk
- Hohe Belastungen der Datenbank, des Servers und des Netzwerks
- Negativ für Gesamtantwortzeit

Stored Procedures/Functions



- Ähnlich zu Prozeduren und Funktionen in z. B. Free Pascal
- Werden in der Datenbank selbst abgespeichert und nicht in der Anwendung
- Prozeduren und Funktionen führen beliebige Befehle aus, wie ein Sub-Programm

Definition einer Procedure

Deklaration (nicht vollständig)

```
1 CREATE PROCEDURE name ([parameter][, parameter]*)
2 BEGIN
3     anweisungsblock
4 END;
5
6 parameter:
7     [IN | OUT | INOUT] name typ
8 typ:
9     jeder gültige MySQL Datentyp
```

- Es gibt keine Rückgabe von Werten
- Dies geschieht über die Parameter (siehe nächste Folie)
- Bei den Parameternamen empfiehlt es sich für die Übersichtlichkeit einen Präfix zu verwenden.
- Zum Beispiel: v_parameter

Parameterarten in Stored Procedures

Art	Präfix	Beschreibung
IN	i	Der Parameter ist eine lokale Variable. Jede Wertänderungen innerhalb der Prozedur bleiben außerhalb der Prozedur unbekannt. Wird kein Parameter angegeben, wird IN angenommen.
OUT	o	Der Parameter wird innerhalb der Prozedur mit einem neuem Wert versehen. Dieser neue Wert kann anschließend außerhalb der Prozedur weiter verarbeitet werden.
INOUT	io	Der Wert des Parameters wird innerhalb der Prozedur verwendet und abgeändert wieder in diesen Parameter abgelegt.

Definition einer Prozedur

- Wenn Sie eine Prozedur wie folgt in Ihr SQL-Abfrage-Fenster eingeben:

Das funktioniert so noch nicht ganz

```
1 CREATE PROCEDURE studyHighSemCount (  
2     IN    sem    INT ,  
3     OUT   studys INT )  
4 BEGIN  
5     SELECT COUNT(*) INTO studys  
6     FROM hochschuldatenbank.studenten  
7     WHERE Semester > sem;  
8 END;
```

- ..., werden Sie merken, dass Ihr SQL-Editor einen/mehrere Syntax-Fehler anzeigt
Warum?
- Ein Semikolon steht für eine abgeschlossene Anweisung und die weiteren Befehle werden nun fehlinterpretiert.

Definition einer Prozedur

- Mit einem **DELIMITER** (Trennzeichen) wird der Abschluss eines Befehls gekennzeichnet
- Das Trennzeichen darf nicht anderweitig innerhalb der Prozedur verwendet werden.

Das funktioniert schon besser

```
1 DROP PROCEDURE IF EXISTS studyHighSemCount;  
2 USE hochschuldatenbank;  
3  
4 DELIMITER //  
5 CREATE PROCEDURE studyHighSemCount( IN    sem    INT,  
6                                     OUT  studys  INT )  
7 BEGIN  
8     SELECT COUNT(*) INTO studys  
9     FROM studenten  
10    WHERE Semester > sem;  
11 END //  
12 DELIMITER ;
```

Aufruf einer Stored Procedure

Prozedur aufrufen

```
1 SET @studys = 0;    -- Lokale Variable vorsichtshalber mit 0  
   initialisieren  
2 CALL hochschuldatenbank.studyHighSemCount(6, @studys);  
3 SELECT @studys as AnzahlStudierende;    -- Variable auslesen
```

Mit folgendem Ergebnis für unsere Beispieldatenbank:

Result
AnzahlStudierende
7

Analog dazu die Definition einer Function

Deklaration (nicht vollständig)

```
1 CREATE FUNCTION name ([p_name typ][, p_name typ]*) RETURNS type
2   characteristics
3 BEGIN
4   anweisungsblock
5   RETURN value;
6 END;
7
8 typ:   jeder gültige MySQL Datentyp
9
10 characteristics:
11   CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA
```

- Bei einer Funktion gibt es nur Eingabe-Parameter
- Dafür muss eine Funktion einen Rückgabewert (**RETURN** value) enthalten
- In manchen DBMSen wird nach den **characteristics** gefragt, welche die Art der Datennutzung in der Anweisung spezifizieren.

Selbiges Beispiel mit Stored Function

Beispiel für das Zählen der Studierenden ab einem bestimmten Semester:

Beispiel: Function

```
1 DROP function IF EXISTS countHighSemStudys;
2
3 DELIMITER //
4 USE hochschuldatenbank//
5 CREATE FUNCTION countHighSemStudys(p_sem INT) RETURNS int
6   READS SQL DATA
7 BEGIN
8   DECLARE v_anzStudys INT;           -- Lokale Variable definieren
9
10  SELECT COUNT(*) INTO v_anzStudys   -- Zuweisung des Ergebnisses
11  FROM hochschuldatenbank.studenten
12  WHERE Semester > p_sem;
13 RETURN v_anzStudys;
14 END ;//
```

Aufruf einer Stored Function

Hier sieht es schon etwas einfacher aus. Das liegt daran, dass es nur eine Rückgabe-Variable gibt. Bei einer **PROCEDURE** können es mehrere sein.

Beispiel: Function

```
1 SELECT countHighSemStudys(6) as AnzahlStudierende;
```

Mit selbigem Ergebnis für unsere Beispieldatenbank:

Result
AnzahlStudierende
7

Weiteres Beispiel

Hinzufügen von Professor*in und Vorlesung inkl. Herstellen der Verknüpfung zwischen beiden Entitäten.

Beispiel: Professor*in und Vorlesung hinzufügen

```
1 CREATE PROCEDURE InsertProfVL (  
2     IN i_ProfName  varchar(255),  
3     IN i_VLName    varchar(255) )  
4 BEGIN  
5     DECLARE v_prof_id INT DEFAULT 0;  
6  
7     INSERT INTO hochschuldatenbank.professoren (Name) VALUE (  
8         i_ProfName);  
9  
10    SET v_prof_id = last_insert_id();  -- Hole letzte angelegte ID  
11  
12    INSERT INTO hochschuldatenbank.vorlesungen (Titel, gelesenVon)  
13        VALUES (i_VLName, v_prof_id);  
14 END;
```

Bedingungen bzw. Verzweigungen

- IF, THEN, ELSE[if] END IF; sollte aus nahezu jeder Programmiersprache bekannt sein.

Beispiel: IF-Abfrage

```
1 CREATE FUNCTION SimpleCompare(n INT, m INT)
2 RETURNS VARCHAR(20) NO SQL
3
4 BEGIN
5     DECLARE s VARCHAR(20);
6
7     IF      (n > m) THEN SET s = '>';
8     ELSEIF (n = m) THEN SET s = '=';
9     ELSE SET s = '<';
10    END IF;
11
12    SET s = CONCAT(n, ' ', s, ' ', m);
13
14    RETURN s;
15 END;
```

Mehrfache Verzweigung mit CASE

- Gleiches gilt für die CASE-Verzweigung

CASE-Syntax in SQL

```
1 CASE spaltenname | ausdruck | variablenname
2     WHEN wert THEN anweisungsblock
3     [WHEN wert THEN anweisungsblock]*
4     [ELSE anweisungsblock]
5 END CASE;
6
7 -- Weitere Alternative der Formulierung
8 CASE
9     WHEN bedingung THEN anweisungsblock
10    [WHEN bedingung THEN anweisungsblock]*
11    [ELSE anweisungsblock]
12 END CASE;
```

Schleifen

Schleifen in SQL

```
1  -- LOOP entspricht for(;;)
2  [labelname:]LOOP
3      anweisungsblock
4  END LOOP[ labelname];
5
6  [labelname:]WHILE schleifenbedingung DO
7      anweisungsblock
8  END WHILE[ labelname];
9
10 [labelname:]REPEAT
11     anweisungsblock
12 UNTIL bedingung END REPEAT[ labelname];
```

Befehle innerhalb der Anweisungsblöcke aller drei Schleifentypen:

- **LEAVE** [labelname]: Verlassen der Schleife (break)
- **ITERATE** [labelname]: Ein weiterer Durchlauf (continue)

Abwägung der Nutzung von *Stored Programs*

- DBMS kann langsamer sein als PHP, Java, etc. bei zahlenverarbeitenden Algorithmen, komplexen String Manipulationen, etc.
- *Stored Programs* punkten hauptsächlich bei Datenbank-Zugriffen durch die kurzen Antwortzeiten, die nicht über das Netzwerk gesendet werden müssen.
- Entwicklungen in unterschiedlichen Bereichen (*Stored Programs*, Geschäftslogik und Anwendungs-Client) können zur Fragmentierung und schlechteren Les- & Wartbarkeit von Quellcode führen.
- *Stored Programs* sind nur bedingt portierbar zwischen unterschiedlichen DBMSen.

Übersicht

1 SQL (weitere Konzepte)

Datenbankadministrationssprache

Prozeduren und Funktionen

Cursors

Behandlung von Ausnahmen

Transaktionen

Datenintegrität

Trigger



Cursors

- „**SELECT INTO** <variable>“ lässt nur ein Ergebniswert zu, welcher in die Variable geschrieben wird
- Soll mehr als eine Ergebnis-Zeile verarbeitet werden, müssen **Cursor** verwendet werden
- Ein **Cursor** ähnelt einem Objekt in C++/Java/etc. auf welches programmatisch zugegriffen werden kann.
- Eigenschaften in MySQL und MariaDB:
 - **Schreibgeschützt:** Arbeiten auf temporären Tabellen, sodass Änderungen nicht in den Tabellen übernommen werden.
 - **Vorwärtsgerichtet:** Man kann immer nur zum nächsten Datensatz gehen und nicht zurück. Datensätze können nicht übersprungen werden.
 - **Serverseitig:** Nur verwendbar innerhalb von Prozeduren. Nicht verwend-/erstellbar aus Client-Anwendungen heraus.

Syntax zur Erstellung von Cursors

Cursors müssen nach der Variablendeklaration erstellt werden.

Definition eines Cursors

```
1 DECLARE cursor_name CURSOR FOR SELECT_statement;
2
3 -- Beispiel (nicht vollständig):
4 CREATE PROCEDURE cursor_demo (in_customer_id INT)
5 BEGIN
6     DECLARE v_customer_id    INT;
7     DECLARE v_customer_name  VARCHAR(30);
8     DECLARE cursor1 CURSOR FOR
9         SELECT in_customer_id, customer_name
10        FROM customers
11        WHERE customer_id=in_customer_id;
12
13 -- Fetching eine Zeile von einem Cursor
14 OPEN cursor1; -- Cursor öffnen
15 FETCH cursor1 INTO v_customer_id, v_customer_name;
16 CLOSE cursor1; -- und wieder schließen
```

Natürlich wollen wir mehrere Ergebnis-Zeilen verarbeiten.

Verarbeiten mehrerer Zeilen

```
1 ...  
2 DECLARE c_dept CURSOR FOR  
3   SELECT department_id  
4   FROM departments;  
5  
6 OPEN c_dept;  
7 dept_cursor: LOOP  
8   FETCH c_dept INTO l_dept_id;  
9 END LOOP dept_cursor;  
10 CLOSE c_dept;  
11 ...
```

- Sieht auf den ersten Blick ganz ordentlich aus
- Aber: Die Schleife würde auf eine Zeile nach der letzten Zeile zugreifen wollen und wirft einen Fehler ("no data to fetch").

Cursor - Korrigierte Variante

```
1 ...
2 DECLARE dept_csr CURSOR FOR
3   SELECT department_id, department_name, location FROM departments;
4 DECLARE CONTINUE HANDLER FOR NOT FOUND SET no_more_departments=1;
5
6 OPEN dept_csr;           -- Öffnen des Cursors
7 dept_loop1: LOOP        -- Iteratives Zeilen laden
8   FETCH dept_csr INTO l_department_id, l_department_name, l_location;
9   IF no_more_departments=1 THEN
10    LEAVE dept_loop1;    -- Falls Fehler geworfen wird, setzt der
                          -- Handler die Variable auf 1 und setzt die Bearbeitung fort
                          -- (CONTINUE) und die Schleife wird verlassen
11  END IF;
12  SET l_department_count=l_department_count+1;
13 END LOOP;
14 CLOSE dept_csr;
15 SET no_more_departments=0;
16 ...
```

Weiteres zu Cursors

- In einer Prozedur können auch mehrere **Cursor** deklariert werden
- Diese können wie in anderen Programmiersprachen verschachtelt sein
- Für den **HANDLER** ist darauf zu achten, dass die „not found“ - Variable zurückgesetzt wird, nachdem die Schleife beendet ist.
- Generell gilt: Falls geschickte SQL-Anweisungen möglich sind, sind diese aus Performance-Gründen der Verwendung von **Cursors** vorzuziehen.

Übersicht

1 SQL (weitere Konzepte)

Datenbankadministrationssprache

Prozeduren und Funktionen

Cursors

Behandlung von Ausnahmen

Transaktionen

Datenintegrität

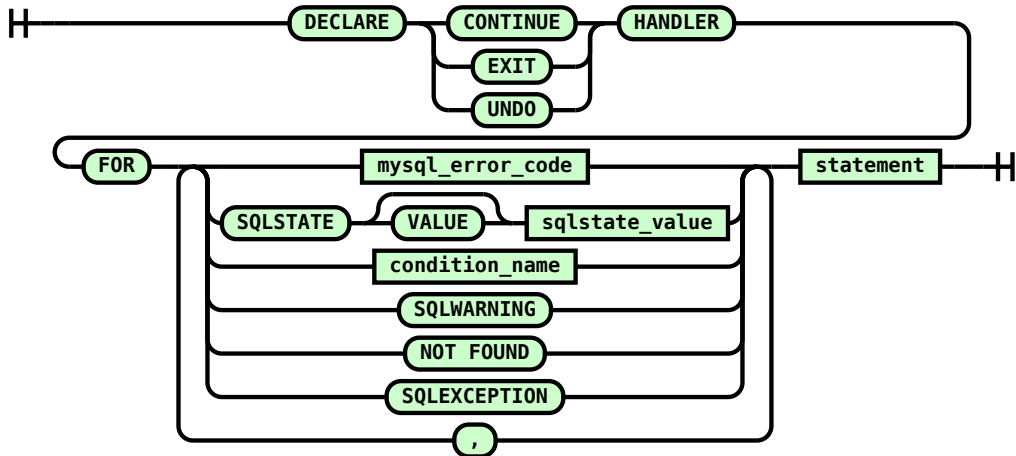
Trigger



Exception Handling

- Sie können noch so gute SQL-Anweisungen programmieren, die in all Ihren Test-Cases fehlerfrei durchlaufen.
- Ihr Benutzer kann Ihr Programm dennoch falsch benutzen.
- Mit dem Ergebnis, dass Ihre SQL-Anweisungen fehlschlagen.
- MySQL und viele andere DBMS'e bieten **exceptions** an, um mit *Fehlern* umzugehen.

Exception Handling - Syntax



Exception Handling - Syntax (2)

- **mysql_error_code**: Ein MySQL-Fehlercode. Bsp.: 1051 spezifiziert: Unbekannte Tabelle. Eine Liste von Fehlercodes befindet sich hier¹.
- **SQLSTATE [VALUE] <sqlstate_value>**: Ein 5-stelliger Code, der einen SQLSTATE Wert kennzeichnet. Bsp.: '42S01' für Unbekannte Tabelle.
- **condition_name**: Mit **DECLARE ... CONDITION ...** lassen sich weitere Fehlerbedingungen mit einem sprechenden Namen versehen. Diese können an dieser Stelle referenziert werden.
- **SQLWARNING**: Kurzform der SQLSTATE Werte, die mit '01' beginnen.
- **NOT FOUND**: Kurzform aller SQLSTATE Werte, die mit '02' beginnen. Insbesondere wichtig für die Verwendung mit **Cursors**, wobei sämtliche „Ende-des-Datensets“-Bedingungen abgefangen werden.
- **SQLEXCEPTION**: Kurzform für alle SQLSTATE Werte, die nicht mit '00', '01', oder '02' beginnen.

¹<https://dev.mysql.com/doc/refman/8.0/en/server-error-reference.html>

Exception Handling - Beispiel

```
1 CREATE PROCEDURE add_department ...
2 BEGIN
3     DECLARE duplicate_key INT DEFAULT 0;
4     /* Duplicate key */
5     DECLARE CONTINUE HANDLER FOR 1062 SET duplicate_key=1;
6     INSERT INTO ...
7
8     IF duplicate_key=1 THEN
9         SELECT 'Fehler beim Einfügen: Doppelter Key' AS "Result";
10    ELSE
11        -- Wenn kein Fehler auftritt, geht's hier weiter.
12    END IF;
13 END //
```

Exception Handling - Beispiel (EXIT-Fall)

```
1 CREATE PROCEDURE add_department ...
2 BEGIN
3     DECLARE duplicate_key INT DEFAULT 0;
4     BEGIN
5         /* 1062 = Duplicate key */
6         DECLARE EXIT HANDLER FOR 1062 SET duplicate_key=1;
7         INSERT INTO ...
8         SELECT ... -- Wenn der Schlüssel bereits vergeben ist, so
                       wird diese Anweisung im Block nicht mehr ausgeführt und er
                       springt weiter in Zeile 10.
9     END;
10
11     IF duplicate_key=1 THEN
12         SELECT 'Fehler beim Einfügen: Doppelter Key' AS "Result";
13     END IF;
14 END //
```

Ähnliche Fehlerbehandlungen

Mehrere ähnliche Fehlerbehandlungen

```
1 DECLARE EXIT HANDLER FOR 1062 SELECT 'Fehler 1062 gefunden';
2 DECLARE EXIT HANDLER FOR SQLEXCEPTION SELECT 'Fehler gefunden';
3 DECLARE EXIT HANDLER FOR SQLSTATE '23000' SELECT 'SQLSTATE 23000';
4
5 INSERT INTO vorlesungen VALUES ('4052', 'Logik', '4', '2125', '5')
    ;
```

- Welcher Handler wird zuerst angesprochen?
- Reihenfolge: Vom spezifischsten zum unspezifischsten:
 - 1 **mysql_error_code**
 - 2 **SQLSTATE**
 - 3 **SQLWARNING** und **SQLEXCEPTION**

Exception Handling - Best practice

- ☒ Behandle Ausnahmen, die nicht vermieden werden können (z. B.: **Cursor**)
- ☒ Sprechende Fehlernamen besser als Fehlercodes
- ☒ Konsistent sein in der Nutzung der Fehlerbehandlung
- ☒ Eher spezifischere Handler verwenden als **SQLEXCEPTION**

Übersicht

1 SQL (weitere Konzepte)

Datenbankadministrationssprache

Prozeduren und Funktionen

Cursors

Behandlung von Ausnahmen

Transaktionen

Datenintegrität

Trigger



Transaktionsverwaltung

Die Verwendung von Transaktionen ist einer der wichtigsten Punkte, in dem sich ein DBMS vom Dateisystem unterscheidet.

Beispiel einer typischen Transaktion in einer Bankanwendung:



- ① Lese den Kontostand von A in die Variable a: `read(A,a);`
- ② Reduziere den Kontostand um 50.- Euro: `a:= a - 50;`
- ③ Schreibe den neuen Kontostand in die Datenbasis: `write(A,a);`
- ④ Lese den Kontostand von B in die Variable b: `read(B,b);`
- ⑤ Erhöhe den Kontostand um 50,- Euro: `b := b + 50;`
- ⑥ Schreibe den neuen Kontostand in die Datenbasis: `write(B, b);`

Operationen auf Transaktions-Ebene

In den klassischen Transaktionssystemen:

- **START TRANSACTION**: Mit diesem Befehl wird der Beginn einer Folge von SQL-Anweisungen gekennzeichnet.
- **COMMIT**: Beendigung der Transaktion. Alle Änderungen werden dauerhaft in die Datenbank festgeschrieben.
- **ABORT**: Selbstabbruch der Transaktion. Das DBMS muss sicherstellen, dass die Datenbasis wieder in den Zustand zurückgesetzt wird, der vor Beginn der Transaktionsausführung existierte.

Transaktionen lassen sich auch als einfache Prozedur oder Funktion realisieren. Dabei ist darauf zu achten, dass **AUTOCOMMIT** in MySQL standardmäßig auf „1“ gesetzt ist. Sprich, jede Anweisung wird direkt ausgeführt. Um bei einer Folge von Anweisungen sicher zu gehen, dass sie erst am Ende final ausgeführt werden, sollte es auf 0 gesetzt werden: (SET autocommit=0;) und am Ende **COMMIT**; aufrufen.

Eigenschaften von Transaktionen: ACID

- **A**tomicity (Atomarität): Alles oder nichts
- **C**onsistency: Konsistenter Zustand der DB → k. Zustand
- **I**solation: Jede Transaktion hat die DB *für sich alleine*
- **D**urability (Dauerhaftigkeit): Änderungen erfolgreicher Transaktionen dürfen nie verloren gehen

Transaktionsverwaltung in SQL

- **COMMIT [WORK]** Die in der Transaktion vollzogenen Änderungen werden – falls keine Konsistenzverletzung oder andere Probleme aufgedeckt werden – festgeschrieben. Das Schlüsselwort **WORK** ist optional.
- **ROLLBACK [WORK]** Zurücksetzen aller Änderungen. Anders als der **COMMIT**-Befehl muss das DBMS die *erfolgreiche* Ausführung eines **ROLLBACK**-Befehls immer garantieren können.
Mit **SAVEPOINT**s lassen sich Sprungmarken setzen, zu denen beim Rollback gesprungen werden kann (**ROLLBACK [WORK] TO [SAVEPOINT]** bezeichner), um bspw. nicht die gesamte Transaktion rückgängig zu machen. Diese sollten allerdings mit Bedacht verwendet werden, da sie schnell zu „Spagetticode“ führen können.

Beispiel aus unserer Hochschuldatenbank

Transaktion zum Auslesen und Aktualisieren der Credits

```
1 START TRANSACTION;  
2 SELECT @CR:=Credit FROM vorlesungen WHERE Titel='Logik';  
3 UPDATE studenten SET ETCS=@CR WHERE MatrNr=24002;  
4 COMMIT;
```

Übersicht

1 SQL (weitere Konzepte)

Datenbankadministrationssprache

Prozeduren und Funktionen

Cursors

Behandlung von Ausnahmen

Transaktionen

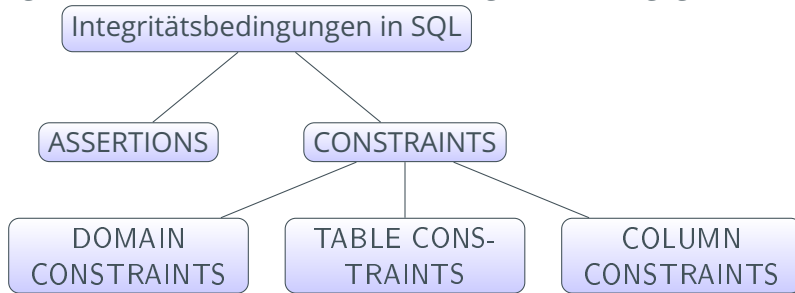
Datenintegrität

Trigger



Integritätsbedingungen in SQL

- Semantische Korrektheit der gespeicherten Daten
- Prüfung aller Daten hinsichtlich ihrer Richtigkeit für die gegebene Anwendung



- **ASSERTIONS:** Formulierung von Einschränkungen von 1-n Tabellen
- **CONSTRAINTS:** Hängen unmittelbar von einer Tabelle ab
 - Schlüssel
 - Beziehungskardinalitäten
 - Attributdomänen
 - Inklusion bei Generalisierung

Integritätstypen

Nachfolgende Abbildung zeigt, welche aus der ER-Modellierung bekannten Integritätsbegriffe durch welche SQL-Konzepte realisiert werden:

**CONSTRAINT / ASSERTION
mit CHECK-Bedingung**



**semantische
Integrität**

**CONSTRAINT mit
PRIMARY KEY**



**Entity-
Integrität**

**CONSTRAINT mit
FOREIGN-KEY
und REFERENCES**



**referentielle
Integrität**

Semantische Integritätsbedingungen

- Wertebereichseinschränkungen
 - ... CHECK (Semester BETWEEN 1 AND 13)
- Aufzählungstypen
 - ... CHECK (Rang IN ('C2', 'C3', 'C4')) ...

Das Universitätsschema mit Integritätsbedingungen

```
1 CREATE TABLE Studenten
2 ( MatrNr    integer primary key,
3   Name      varchar(60) NOT NULL,
4   Semester  integer CHECK (Semester BETWEEN 1 AND 13)
5 );
6
7 CREATE TABLE Professoren
8 ( PersNr    integer primary key,
9   Name      varchar(60) NOT NULL,
10  Rang      character(2) CHECK (Rang IN ('C1', 'C2', 'C3', 'C4')),
11  Raum      integer UNIQUE
12 );
```

Referentielle Integrität

Definition

Wenn es zu jedem Fremdschlüsselwert einen passenden Primärschlüsselwert gibt, ist die *Verknüpfung/Referenz integer*. Ist diese Bedingung nicht erfüllt, spricht man von einer *verletzten referenziellen Integrität*.

Referentielle Integrität in SQL

- Primärschlüssel: **primary key**
- Fremdschlüssel: **foreign key**
- (Kandidaten-/Sekundärschlüssel: **unique**)
- Beispiel:

References

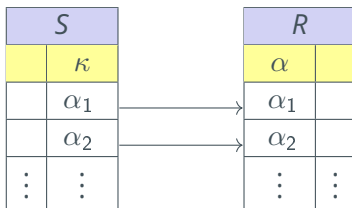
```
1 CREATE TABLE R
2     ( a integer primary key,
3       ... );
4 CREATE TABLE S
5     ( ..., b integer references R );
```

Einhaltung referentieller Integrität

Bedeutung der Fehlerkorrekturoptionen	
Befehl	Beschreibung
ON DELETE	Die definierte Fehlerkorrekturoption wird ausgeführt, wenn der referenzierte Master-Datensatz gelöscht wird.
ON UPDATE	Die definierte Fehlerkorrekturoption wird ausgeführt, wenn der Wert des referenzierten Schlüsselattributs in der Master-Tabelle geändert wird.
CASCADE	Die Detaildatensätze, deren Fremdschlüssel den manipulierten Master-Datensatz referenzieren, werden ebenfalls gelöscht (kaskadierendes Löschen) bzw. deren Schlüsselwert wird entsprechend geändert (kaskadierendes Ändern).
SET DEFAULT	Die Fremdschlüsselspalten des Detaildatensatzes, die den manipulierten Master-Datensatz referenzieren, werden auf den für diese Spalte definierten DEFAULT-Wert gesetzt.
SET NULL	Die Fremdschlüsselspalten des Detail-Datensatzes, die den manipulierten Master-Datensatz referenzieren, werden auf NULL gesetzt.
NO ACTION	Auf den Integritätsfehler wird je nach Prüfungszeitpunkt mit dem Zurückrollen der gesamten Transaktion bzw. der fehlerhaften DML-Anweisung reagiert (Reaktion wie bei den anderen CONSTRAINTS auch).
RESTRICT	Diese Option ist analog zur NO ACTION-Funktionalität zu sehen, mit dem kleinen Unterschied, dass bei RESTRICT vorübergehend während der Ausführung der UPDATE- oder DELETE-Anweisung Verletzungen des Fremdschlüssels hingenommen werden, solange am Ende die Bedingungen wieder erfüllt sind.

Einhaltung referentieller Integrität

- Relevant bei Änderungen wie **UPDATE** oder **DELETE**



Änderungsoperationen

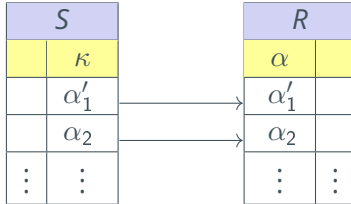
```
1 UPDATE R
2 SET  $\alpha = \alpha_s1$ 
3 WHERE  $\alpha = \alpha_1$ ;
```

Änderungsoperationen

```
1 DELETE FROM R
2 WHERE  $\alpha = \alpha_1$ ;
```

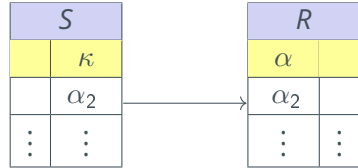
Kaskadieren

- Automatische Änderungen und Löschweitergabe



Änderungswweitergabe

```
1 CREATE TABLE S
2 ( ...,
3    $\kappa$  integer references R
4   on update cascade );
```

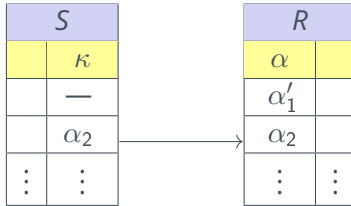


Löschweitergabe

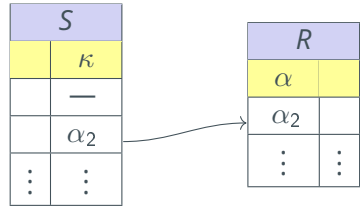
```
1 CREATE TABLE S
2 ( ...,
3    $\kappa$  integer references R
4   on delete cascade );
```

Auf NULL setzen

- Automatische Änderungsweitergabe -> SET NULL



```
1 CREATE TABLE S
2 ( ...,
3   κ integer references R
4   on update set null );
```



```
1 CREATE TABLE S
2 ( ...,
3   κ integer references R
4   on delete set null );
```

Beispiele

- Auswirkung: Löscht alle Vorlesungen V zu Prof. X, wenn Prof. X gelöscht wird.

Cascade

```
1 CREATE TABLE Vorlesungen
2 ( ...,
3 gelesenVon integer
4   references Professoren
5   ON DELETE CASCADE);
```

Beispiele

- Auswirkung: Löscht alle auf Vorlesung V verweisende Einträge in der „hören“-Tabelle , wenn Vorlesung V gelöscht wird (ggf. als Folge der Löschung von Prof. X).

CASCADE

```
1 CREATE TABLE hoeren
2 ( ...,
3   VorlNr integer,
4   references Vorlesungen ON DELETE CASCADE
5 );
```

Das Unischema mit ref. Integritätsbedingungen

Beispiel: Assistenten und Vorlesungen

```
1 CREATE TABLE Assistenten
2 ( PersNr          integer primary key ,
3   Name            varchar(50) NOT NULL ,
4   Fachgebiet      varchar(50) ,
5   Boss            integer ,
6   foreign key     (Boss) references Professoren (PersNr) ON DELETE
      SET NULL ON UPDATE SET NULL
7 );
8
9 CREATE TABLE Vorlesungen
10 ( VorlNr          integer primary key ,
11  Titel            varchar(80) ,
12  SWS              integer ,
13  gelesenVon       integer references Professoren ON DELETE SET NULL ON
      UPDATE SET NULL
14 );
```

Das Unischema mit ref. Integritätsbedingungen (2)

Beispiel: hören und voraussetzen

```
1 CREATE TABLE hoeren
2 ( MatrNr  integer NOT NULL references Studenten ON UPDATE CASCADE
   ON DELETE CASCADE,
3  VorlNr  integer NOT NULL references Vorlesungen ON UPDATE
   CASCADE ON DELETE CASCADE,
4  primary key (MatrNr, VorlNr)
5 );
6
7 CREATE TABLE voraussetzen
8 ( Vorgaenger integer NOT NULL references Vorlesungen ON DELETE
   CASCADE ON UPDATE CASCADE,
9  Nachfolger integer references Vorlesungen ON DELETE CASCADE ON
   UPDATE CASCADE,
10 primary key (Vorgaenger, Nachfolger));
```

Das Unischema mit Integritätsbedingungen

Beispiel für semantische und referentielle Bedingungen

```
1 CREATE TABLE pruefen
2 ( MatrNr integer references Studenten ON DELETE CASCADE,
3   VorlNr integer references Vorlesungen,
4   PersNr integer references Professoren ON DELETE SET NULL,
5   Note numeric (2,1) check (Note BETWEEN 1.0 AND 5.0),
6   primary key (MatrNr, VorlNr));
```

Der Integritätsmonitor

Das Datenbankmanagementsystem verfügt über eine zentrale Komponente zur Verwaltung und Prüfung der Integritätsbedingungen, den so genannten Integritätsmonitor.

Seine Aufgaben sind:

- die Verwaltung der in der Datenbank abgelegten Integritätsbedingungen,
- das Erkennen von prüfungsrelevanten Ereignissen (**INSERT, UPDATE, DELETE**),
- das Ermitteln der von diesen Ereignissen betroffenen Integritätsbedingungen,
- das Auswerten (Kontrollieren) der Bedingungen,
- die Interpretation der Ergebnisse der Integritätskontrolle sowie
- die Reaktion auf diese Ergebnisse (Zurückrollen, Fehlerkorrektur ...).

Übersicht

1 SQL (weitere Konzepte)

Datenbankadministrationssprache

Prozeduren und Funktionen

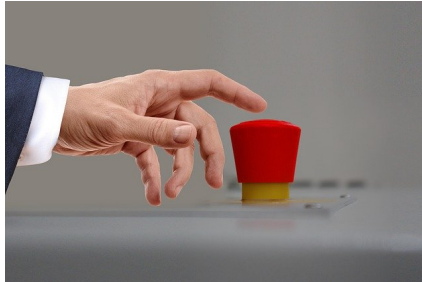
Cursors

Behandlung von Ausnahmen

Transaktionen

Datenintegrität

Trigger



Trigger

- Datenbank Trigger gehören zur Familie der *Stored Programs*
- Wie der Name bereits sagt, werden Trigger durch Tabellen-Aktionen ausgelöst
- Trigger reagieren auf folgende DML-Anweisungen: **INSERT, UPDATE, DELETE**
- Werden oft für folgende Szenarien verwendet:
 - Sicherstellung von Daten-Integrität
 - Automatisierung von Operationen in der Datenbank (z. B.: Denormalisierung)
 - Logging sämtlicher Prozesse
- Eigenschaften:
 - Keine zwei **Trigger** für das gleiche Ereignis mit gleicher Ausführungszeit derselben Tabelle.
 - Der Triggername ist in der Datenbank einmalig.
 - Der Trigger hat keine Übergabeparameter.
 - Der Trigger liefert nichts zurück.

Trigger

+ Vorteile

- Verarbeitung an zentraler Stelle in der Datenbank - und nicht (fehleranfällig) in den Anwendungsprogrammen

- Nachteile

- Die Syntax der Trigger ist DBMS-spezifisch
- Auch die Verarbeitung ist unterschiedlich
- Werden Trigger innerhalb der initiiierenden Transaktion ausgeführt oder bilden sie eine eigene Transaktion?

Trigger: Syntax

Trigger Definition

```
1 CREATE [DEFINER={user|CURRENT_USER}] TRIGGER trigger_name
2   {BEFORE|AFTER}
3   {UPDATE|INSERT|DELETE}
4   ON table_name
5   FOR EACH ROW
6   trigger_statements
```

- Der signifikanteste Unterschied zwischen **BEFORE** und **AFTER** liegt darin, dass bei einem **AFTER**-Trigger die Einzufügenden bzw. die zu Aktualisierenden Werte nicht mehr verändert werden können.
Wenn die DML-Anweisung gefeuert ist, ist es zu spät für jegliche Änderungen.
- Mit **NEW** und **OLD** lässt sich im Prozess des Einfügens bzw. des Ändern & Löschens auf die jeweils neu eingegebenen Werte bzw. die bereits existierenden zugreifen.

Trigger Beispiel

AFTER Trigger kann NEW-Werte nicht modifizieren

```
1 DELIMITER //
2 USE hochschuldatenbank //
3 CREATE TRIGGER creditUpdater AFTER UPDATE ON vorlesungen FOR EACH
  ROW
4 BEGIN
5     DECLARE someVar INT;
6
7     IF (NEW.Credit < 0) THEN
8         SET NEW.Credit = NULL;
9     END IF;
10
11 END //
```

Lässt sich mit folgendem Fehler nicht anlegen: Error Code: 1362. Updating of NEW row is not allowed in after trigger

Trigger für Logging

- Viele Organisationen und Regularien fordern Transparenz und Nachverfolgbarkeit von Datenbewegungen in Applikationen.

Beispiel für Logging

```
1 CREATE TRIGGER tr_Blutwerte_Update
2   AFTER UPDATE ON blutwerte FOR EACH ROW
3 BEGIN
4   INSERT into transaction_log
5     (user_id, description)
6   VALUES (user(),
7           CONCAT('Änderung des Blutwertes ',
8                 NEW.blutwert_ID, ' FROM ', OLD.blutwerte,
9                 ' TO ', NEW.blutwerte
10          )
11         );
12 END;
```

Trigger für Validierung

- Auch genannt: *check constraint triggers*
- Prüft erlaubte Werte in Spalten
- Validierung zwischen mehreren Spalten bzw. Tabellen
- Erweiterte referentielle Integrität

Beispiel für Logging

```
1 CREATE TRIGGER tr_Blutwerte
2   BEFORE UPDATE ON blutwerte
3   FOR EACH ROW
4   BEGIN
5     -- Blutwerte kann nicht negativ sein.
6     IF (NEW.Blutwert < 0) THEN
7       -- Warnung! Nicht implementiert in MySQL 5.0...
8       SIGNAL SQLSTATE '80000'
9       SET MESSAGE_TEXT='Blutwert darf nicht kleiner 0 sein.';
10    END IF;
11  END;
```

Trigger Overhead

- Trigger erzeugen immer Overhead für alle DML-Anweisungen
- Besonders, weil MySQL Trigger **FOR EACH ROW** ausführt
- Daher: Vermeiden von umfangreichen SQL-Anweisungen

Trigger auflisten

- Falls der SQL-Editor keine Funktion zur Auflistung von Trigger besitzt sind diese in folgender Tabelle zu finden: **INFORMATION_SCHEMA.TRIGGERS**
- Diese können ebenfalls mit dem **SHOW TRIGGERS [FROM db_name]** Befehl aufgerufen werden
- Eine weitere Alternative ist der SQL-DUMP, welche diese ebenfalls enthalten.

Besser leserlich:

```
1 SELECT CONCAT('CREATE TRIGGER ',
2               t.TRIGGER_SCHEMA, '.', t.TRIGGER_NAME, ' ',
3               t.ACTION_TIMING, ' ',
4               t.EVENT_MANIPULATION, ' ON ',
5               t.EVENT_OBJECT_SCHEMA, '.', t.EVENT_OBJECT_TABLE,
6               ' FOR EACH ROW ', t.ACTION_STATEMENT, '//',
7               ) AS TriggerListe
8 FROM INFORMATION_SCHEMA.TRIGGERS AS t;
```

Zusammenfassung

- Sie wissen nun...
 - ... welche Administrationsfunktionen es gibt.
 - ... was Integritätsbedingungen sind, welche Typen es gibt und wie diese definiert sind
 - ... wie Fremd- und wie Primärschlüssel in SQL programmiert werden
 - ... was Referentielle Integrität bedeutet und wie man Sie in SQL programmiert sowie wie man eine Löscho- und Änderungsweitergabe programmiert
 - ... wie man statische Integritätsbedingungen formuliert (**CHECK**-Anweisung)
 - ... wie man dynamische Integritätsbedingungen programmiert (**Trigger**)
 - ... wie Ausnahmen bei *Stored Programs* implementiert werden
 - ... wie Transaktionen entwickelt werden

Weiterführende Literatur

- Ralf Adams. SQL - Der Grundkurs für Ausbildung und Praxis. Hanser Verlag.
([https://th-owl.digibib.net/search/katalog/record/\(DE-605\)HT020278109](https://th-owl.digibib.net/search/katalog/record/(DE-605)HT020278109))
- Guy Harrison, Steven Feuerstein. MySQL Stored Procedures Programming. O'Reilly Media. ISBN: 978-0-596-10089-6.

Vielen Dank!